

Native QA — feature branch, 2026-09-10

Verdict: not ready to merge. Native testing found two functional blockers and reproducible editor/terminal runtime errors despite passing unit tests and the earlier mocked-browser checks.

Build: feat/audit-reliability-fixes, commit 8c5a482, Windows native debug executable. No application-source changes or PR updates were made during this QA pass.

Test environment

- Connected Playwright over CDP to a separate native WebView2 instance, using its real Rust IPC, filesystem, PTYs, and OS windows.
- Used a separate WebView user-data folder and disposable repositories under `.cache/native-qa-*`.
- Left the user's installed and previously launched feature instances alone. The QA instance's periodic session-save and restore-row-clear calls were suppressed to avoid overwriting the shared saved workspace. Other tested commands reached the real backend.
- Opened an agent session without sending a prompt to establish the application's required filesystem grant. Shell-only sessions intentionally do not grant file access, so their initial denied file operations were excluded from the functional verdict.
- Closed all QA-created terminals and detached windows after testing. Fixture files, screenshots, and raw results remain available for reproduction.

Confirmed failures

1. High — Native dirty-file close still discards edits without waiting for consent

Location: `src/components/TerminalTabs.tsx:58-61`, used by mouse, middle-click, and keyboard close handlers.

Reproduction:

1. Open a file in a trusted repository and edit it with auto-save disabled.
2. Focus its dirty-file close control and press Enter.
3. The tab disappears without an accepted discard decision. Its edits are absent from disk.

On this native build, the injected `window.confirm` is an async function returning a Promise. The synchronous `!window.confirm(...)` condition treats that Promise as truthy and proceeds directly to `closeFileTab`.

Instrumentation that preserved the original function recorded:

```
{
  "calls": [
    {"message": "Discard unsaved changes in other.ts?", "isPromise": true},
    {"error": "dialog.confirm not allowed. Command not found"}
  ],
  "stillOpen": false
}
```

The native override invokes `plugin:dialog|confirm`, which also fails in this runtime. The fix needs an explicitly awaited supported dialog API or an application confirmation modal, with failure keeping the tab open. The earlier browser/unit tests used a synchronous confirmation function and missed the native behavior.

2. High — Local JavaScript-module previews fail to start

Locations: `src/components/PreviewPanel.tsx:153` and `165`.

Static HTML preview now loads, confirming the CSP fix works. However, the `iframe` sandbox omits `allow-same-origin`, giving the document an opaque origin. A local page with a normal `<script type="module" src="/app.js">` cannot load that same-server module unless the server explicitly accepts the opaque origin.

Native reproduction served a minimal HTML/module application on loopback without CORS overrides. The page stayed at "Module pending" and logged:

```
Access to script at 'http://127.0.0.1:57017/app.js' from origin 'null'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header
is present on the requested resource.
```

Iframe restrictions need to support intended development applications while retaining origin isolation from Agentrium and its IPC. Broadly removing the sandbox is not an appropriate fix.

3. Medium — Closing a diff editor raises model-disposal errors

Integration location: `src/components/FileEditorView.tsx:208`.

Open a diff tab and close it. The actual Monaco integration emits:

```
TextModel got disposed before DiffEditorWidget model got reset
```

The stack passes through `@monaco-editor/react` cleanup, `TextModel.dispose`, and the diff widget's model listener. This reproduced both in the main scenario and an isolated follow-up. Diff editing and saving succeeded before close; an application-wide crash or data loss from this error was not observed. Separate canceled errors also appeared during Monaco teardown.

4. Medium — Terminal lifecycle emits uncaught renderer errors

Integration location: `src/components/TerminalView.tsx`; observed stack is inside `xterm`.

Native session creation, transfer, and cleanup repeatedly logged:

```
TypeError: Cannot read properties of undefined (reading 'dimensions')
    at get dimensions (@xterm_xterm.js)
    at Viewport.syncScrollArea (@xterm_xterm.js)
```

PTY input/output and transfers still worked. The precise disposal/rendering race needs investigation; this QA pass does not establish a terminal crash or lost output.

Passing checks

Workflow	Result
Native shell PTY input/output	Echo marker appeared in xterm.
Unborn Git repository	Native changes command recognized the repository and listed files.
Spaces/Unicode in Git paths	Literal filenames survived native status and staging; inspected the actual Git index.
Discard directory junction	Link removed; tracked target and its contents preserved.
Editor preferences	Changing font size through Settings rendered 24px text in Monaco.
Auto-save on tab switch	Independent test wrote keyboard-entered changes to disk.
Diff editing/saving	Modified-side edits saved to disk; font preference applied.
Static localhost preview	Loaded in the actual native preview panel.
Native tear-off	Destination adopted the same backend terminal before source removal.
Return to main	Original PTY returned successfully.
Transfer timeout	Missing destination timed out while retaining the source tab.
Same-directory layout keys	Two shells had distinct saved tab identities.
Native restore handler	Seeded saved configs, clicked the actual Restore button, and verified two same-directory shells reopened in separate native windows with distinct new PTY IDs and correct labels.

The initial auto-save failure was downstream of the dirty-close bug deleting the edited tab. Auto-save itself passed when exercised independently. The restore check used saved configs from real native terminals, but did not restart the application or modify the user's persisted restore row. Actual agent conversation resumption and macOS behavior were not tested.

Evidence

The following artifacts are local-only and are not included in the repository; key observations and error excerpts are preserved above.

- Main scenarios: `../.cache/native-qa-fwcbgx/results.json`
- Isolated follow-up and error stacks: `../.cache/native-qa-fwcbgx/followup.json`
- Native restore result: `../.cache/native-qa-fwcbgx/restore-result.json`
- Module preview screenshot: `../.cache/native-qa-fwcbgx/preview.png`

- Local QA runners: `.cache/native-qa.cjs`, `.cache/native-qa-followup.cjs`, `.cache/native-qa-restore.cjs`

These native findings supersede the earlier audit's claim that all nine fixes were fully verified. In particular, dirty-file closing and preview compatibility remain incomplete.